

AGENTS

Helix Cluster OS — AGENTS.md

Field	Value
Revision	2
Created	2026-05-30
Last modified	2026-06-02T09:49:46Z
Status	active

INHERITED FROM HelixConstitution/ AGENTS.md

All rules in HelixConstitution/AGENTS.md (and the HelixConstitution/Constitution.md and HelixConstitution/CLAUDE.md it references) apply unconditionally. The project-specific rules below extend them.

Project-Specific Extensions

Agent Context

- This is a distributed cluster operating system project (Helix Cluster OS).
- Agents working on this project **MUST** understand the seven-layer architecture (L0-L7).
- Agents **MUST** respect the cross-platform parity rule (PCS-1): every feature needs Linux, macOS, and Windows/WSL2 implementations.
- Agents **MUST** respect the GPU backend coverage rule (PCS-2): all 4 vendor backends require real-hardware testing.

CodeGraph Integration

- CodeGraph (@colbymchenry/codegraph) is installed and indexed.
- Agents **SHOULD** query CodeGraph via MCP for symbol resolution across the project and own-org submodules.
- See docs/CODEGRAPH.md for integration details.

Critical Base Rules Restated

- Anti-bluff covenant binds every test (Constitution §11.4).
- Mutation-paired gates mandatory per §1.1.
- Credentials NEVER tracked; pre-store leak audit per §11.4.10.A.
- Subagent-driven-by-default per §11.4.20.
- Fetch-before-edit per §11.4.37.

AGENT-1: End-User Usability Guarantee (§7.1 + §11.4.39 Reinforcement)

Forensic anchor — verbatim user mandate: > “We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can’t be used! This **MUST NOT** be the case and execution of tests and Challenges **MUST** guarantee the quality, the completion and full usability by end users of the product!”

Operative rules for coding agents: 1. **Tests are necessary but NOT sufficient.** Every feature you implement **MUST** have: - Unit tests (with paired mutation per §1.1) - Integration tests against REAL services (not mocks) - End-to-end tests that exercise the feature as an end user would - Challenge validation via HelixQA framework 2. **A test that passes on a non-functional feature is a PASS-bluff** — severity equivalent to §7.1 violation. If you encounter this, you **MUST** fix the test or the feature. 3. **Every test MUST prove the feature works for end users**, not just that code executes without panic. Verify actual behavior, not just absence of errors. 4. **Challenges (HelixQA) are bound equally** — a Challenge PASS on a broken feature is the same class of defect as a unit test PASS on broken code. 5. **No mock-only validation** for features that claim real-world operation. Mocks are permitted **ONLY** in unit tests per §11.4.27. 6. **Sink-side evidence required:** Every feature closure **MUST** include captured evidence (screenshot, log, metrics) proving end-user-visible operation. Do not mark tasks complete without it.

AGENT-2: Documentation & Materials Continuous-Sync Guarantee (§11.4.106 docs-chain + family)

Forensic anchor — verbatim user mandate: > “while we continuously change the project ... whatever we do on project or any of its services, components, architecture or anything else **MUST** trigger proper updateing of main README document, all project documentation, user guides, manuals, webiste(s) (if any), diagrams, graphs, schemes, SQL defintions and all other related materials! ... Connect all this with docs_chain Submodule and make sure that all this is achieved out of the box, never forgotten or skipped and all docs and materials maintained regularly!”

Restates + cites the binding master rule (Constitution **§11.4.106 docs-chain-mechanical-sync** + anchor family §11.4.12/45/53/56/57/59/60/65/86/93/95, §12.10, §11.4.44; “No documentation ever can be out of sync with its codebase”). Per §11.4.35 inheritance.

Operative rules for coding agents: 1. **Every change carries its docs.** In the same work unit as any code/service/component/architecture/schema/script change, update or author every affected material — main README.md, all docs/**, user guides, manuals, website(s), diagrams/graphs/schemes, SQL/schema definitions — and **ALL** their exports (md → html/pdf/docx). Shipping code with stale docs is a release

blocker. 2. **Drive sync through docs_chain, out of the box.** Use the docs_chain engine (§11.4.106) with chains registered in .docs_chain/contexts/*.yaml; never hand-maintain exports, never use retired ad-hoc scripts. New materials are registered in a context in the same change so enforcement is automatic. 3. **No escape hatch.** No --skip-docs-chain/--fake-transform; conflicts surface (no silent merge); absent tools yield a typed ToolAbsentError + honest SKIP, never a fake PASS. Evidence at qa-results/docs_chain/<run-id>/. 4. **Maintained regularly** — documentation sync runs every wave as a standing parallel work stream, subagent-driven where useful; a stale doc is treated like a failing test.